

FitFinderAi

Intelligent Candidate Discovery & Ranking

Redrob Hackathon — Methodology & Architecture

Team FitFinderAi • Kaanchan Krishna

The Problem

Why keyword matching fails

- ❗ Recruiters review 100s of profiles, still miss the right person.
- ❗ Keyword filters can't understand contextual fit or career arcs.
- ❗ Good candidates get buried; keyword-stuffers rise to the top.
- ❗ Behavioral signals (responsiveness, completion rates) are ignored.
- ❗ No way to catch honeypot / fabricated profiles automatically.

Our approach

- ✅ **Semantic Understanding**
Dense vector embeddings capture meaning, not keywords.
- ✅ **Behavioral Intelligence**
Redrob platform signals measure real-world engagement.
- ✅ **Strict Guardrails**
Honeypot detection + JD red-flag disqualifications.
- ✅ **Blazing Speed**
100K candidates ranked in < 10 seconds on CPU.

Two-Phase Architecture

PHASE 1 — OFFLINE

- 1 Load 100K candidates from JSONL
- 2 Extract text: Title → Skills → Career History
- 3 Embed via all-MiniLM-L6-v2 (384-dim)
- 4 Normalize vectors for cosine similarity
- 5 Build FAISS IndexFlatIP (Inner Product)
- 6 Persist index + metadata pickle to disk

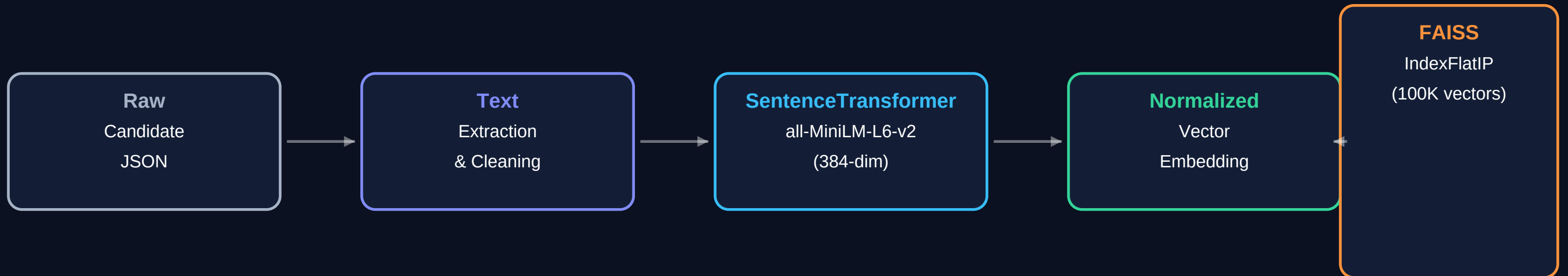
FAISS

Index

PHASE 2 — ONLINE (< 10s)

- 1 Parse & embed the Job Description
- 2 Query FAISS → Top 10,000 semantic matches
- 3 Apply hard filters (YoE, Honeypots, Red Flags)
- 4 Compute behavioral score from Redrob signals
- 5 Fuse: 60% Semantic + 40% Behavioral
- 6 Output Top 100 ranked candidates to CSV

Semantic Embedding Pipeline



Key Design Decision: Text Ordering

- The model has a 256-token context window. We deliberately order text as: Title → Summary → Skills → Career History.
- This ensures high-value skill keywords are always embedded before potential truncation of verbose job descriptions.
- Embeddings are L2-normalized so FAISS Inner Product = Cosine Similarity. No post-processing required.
- Batch encoding at 256 samples/batch with progress bar for the full 100K corpus (~4 min on Apple M-series).

Guardrails & Honeypot Detection

Honeypot Detection

'Expert' proficiency with 0 months duration

→ **REJECT**

Skill duration > total YoE + 5 years

→ **REJECT**

Single job > total YoE + 2 years

→ **REJECT**

Hard Boundary Filters

Years of Experience < 4

→ **REJECT**

Years of Experience > 15

→ **REJECT**

Sweet-spot boost: 5-9 YoE

→ **+0.05 bonus**

JD Red Flag Disqualifications

Consulting-only career
(TCS/Wipro/Infosys...)

→ **REJECT**

Pure researcher without Vector DB
experience

→ **REJECT**

LangChain-only without model/eval
knowledge

→ **REJECT**

Hybrid Scoring Engine

$$\text{Final Score} = \text{Semantic} \times 0.6 + \text{Behavioral} \times 0.4 + \text{YoE Bonus}$$

60% SEMANTIC MATCH

Source	FAISS cosine similarity (Inner Product on normalized vectors)
Input	Candidate career corpus vs. curated JD query embedding
Model	all-MiniLM-L6-v2 — 384 dimensions, 256 token context
Why it works	Captures meaning: 'ML Engineer' matches 'Machine Learning Developer' even without exact keyword overlap

40% BEHAVIORAL SIGNALS

Recruiter Response Rate	×40 weight
Interview Completion Rate	×30 weight
Profile Completeness	×10 weight
GitHub Activity Score	×10 weight
Open to Work Flag	+5 flat bonus
Notice Period > 60 days	-10 penalty

Performance & Constraints

100,000

Candidates
Searched

< 10 sec

End-to-End
CPU Runtime

~500 MB

Peak RAM
Usage

100%

Submission
Validated

Competition Constraint Compliance

- ✓ 5-minute wall-clock limit
- ✓ CPU-only (no GPU)
- ✓ No network calls at runtime
- ✓ 16 GB RAM cap
- ✓ Exactly 100 ranked candidates
- ✓ Honeypot detection

PASS — executes in ~10 seconds

PASS — runs on Apple M-series CPU

PASS — all data is local/precomputed

PASS — peak usage ~500 MB

PASS — validated by validate_submission.py

PASS — multi-signal heuristic detection

Thank You

Built with semantic intelligence, not keyword tricks.

Kaanchan Krishna

krishnakaanchan@gmail.com • +91 90442 77353

github.com/krishnakaanchan-png/ai_candidate_ranker